

Database Scaling

1.DATABASE SCALING PATTERNS

WHAT WILL YOU LEARN?

- HOW TO SCALE A DATABASE STEP BY STEP.
- WHEN TO USE WHICH SCALING OPTION.
- WHICH OPTION IS PRACTICAL AT EACH STAGE OF BUSINESS.

CASE STUDY: CAB BOOKING APP

- A VERY SMALL STARTUP.
- AROUND 10 CUSTOMERS.
- ONE SMALL DATABASE MACHINE STORES EVERYTHING (USERS, TRIPS, LOCATIONS, BOOKINGS, HISTORY).
- ONLY 1 BOOKING EVERY 5 MINUTES.

APP BECOMES POPULAR – PROBLEMS START

- BOOKINGS INCREASE TO 30 PER MINUTE.
- DATABASE BECOMES SLOW.
- API RESPONSE TIME INCREASES.
- DEADLOCKS, STARVATION, FAILED TRANSACTIONS.
- APP FEELS SLOW AND USERS GET ANGRY.

IS THERE A SOLUTION?

- YES, WE NEED PERFORMANCE OPTIMIZATION.
- WE ALSO NEED TO SCALE THE SYSTEM STEP BY STEP.

PATTERN 1: QUERY OPTIMISATION & CONNECTION POOL

- CACHE FREQUENTLY USED DATA (USER PROFILE, BOOKING HISTORY, PAYMENT HISTORY).
- USE DATABASE REDUNDANCY OR EVEN NoSQL WHERE POSSIBLE.
- USE CONNECTION POOLING TO REUSE DATABASE CONNECTIONS.
- MULTIPLE APP THREADS SHARE SAME DB CONNECTION.
- COST IS LOW, BENEFIT IS HIGH AT THIS STAGE.
- BUSINESS EXPANDS TO ONE MORE CITY → ~100 BOOKINGS PER MINUTE.

PATTERN 2: VERTICAL SCALING (SCALE UP)

- UPGRADE THE EXISTING MACHINE.
- DOUBLE THE RAM, FASTER SSD, BETTER CPU.
- EASY AND QUICK SOLUTION.

- CHEAP ONLY TILL A LIMIT.
- AFTER A POINT, COST INCREASES A LOT.
- BUSINESS EXPANDS TO 3 MORE CITIES → ~300 BOOKINGS PER MINUTE.

PATTERN 3: CQRS (COMMAND QUERY RESPONSIBILITY SEGREGATION)

- ONE MACHINE CAN'T HANDLE BOTH READS AND WRITES.
- SEPARATE READ AND WRITE WORKLOADS.
- ONE PRIMARY MACHINE FOR WRITE.
- MULTIPLE REPLICAS MACHINES FOR READ.
- READ QUERIES GO TO REPLICAS.
- WRITE QUERIES GO TO PRIMARY.
- BUSINESS EXPANDS TO 2 MORE CITIES.
- PRIMARY BECOMES WRITE BOTTLENECK.
- REPLICA LAG AFFECTS USER EXPERIENCE.

PATTERN 4: MULTI-PRIMARY REPLICATION

- WHY ONLY PRIMARY SHOULD HANDLE WRITES?
- ALLOW WRITES ON MULTIPLE NODES.
- EACH NODE ACTS AS PRIMARY AND REPLICA.
- DATA IS SYNCED IN A CIRCULAR RING.
- WRITE TO ANY NODE.
- READ FROM FASTEST RESPONDING NODE.
- SCALE TO 5 MORE CITIES.
- SYSTEM STRUGGLES AGAIN (~50 REQUESTS/SECOND).

PATTERN 5: PARTITIONING BY FUNCTIONALITY

- SPLIT DATA BASED ON PURPOSE.
- LOCATION DATA IN SEPARATE DATABASE.
- USER DATA IN ANOTHER DATABASE.
- BOOKINGS IN ANOTHER DATABASE.
- EACH DATABASE CAN HAVE ITS OWN SCALING STRATEGY.
- APPLICATION LAYER HANDLES DATA JOINING.
- PLANNING TO EXPAND TO OTHER COUNTRIES.

PATTERN 6: HORIZONTAL SCALING (SCALE OUT)

- USE SHARDING.
- MULTIPLE MACHINES WITH SAME DB STRUCTURE.
- EACH MACHINE STORES ONLY PART OF DATA.
- DATA LOCALITY IS IMPORTANT.
- EACH SHARD CAN HAVE ITS OWN REPLICAS.
- FAILURE IN ONE SHARD DOES NOT BREAK ENTIRE SYSTEM.

- SHARDING IS HARD BUT VERY POWERFUL.
- BUSINESS SCALES ACROSS CONTINENTS.

PATTERN 7: DATA CENTRE WISE PARTITION

- USERS FROM DIFFERENT CONTINENTS FACE HIGH LATENCY.
- DEPLOY DATABASES IN MULTIPLE DATA CENTRES.
- ROUTE USERS TO NEAREST DATA CENTRE.
- ENABLE CROSS DATA CENTRE REPLICATION.
- HIGH AVAILABILITY AND DISASTER RECOVERY.
- SYSTEM IS NOW ENTERPRISE-GRADE.
- READY FOR IPO 😊